

11주차_6장(Word2Vec~)_자연어 처리와 컴퓨터비전 심층학습

1. Word2Vec

- Word2Vec: 분포 가설 기반의 임베딩 모델
 - 분포 가설: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미를 가질 가능성이 높다
 - 단어 간의 **동시 발생 확률 분포**를 이용해 단어 간의 유사성 측정
- 가정을 통해 단어의 분산 표현 학습
 - 분산 표현: 단어를 고차원 벡터 공간에 매핑하는 것
 - 유사한 문맥에서 등장하는 단어는 비슷한 위치를 갖게 됨

[단어 벡터화]

- **희소 표현** (sparse representation)
 - 대부분의 벡터 요소가 0으로 표현
 - 공간적 낭비 발생
 - 벡터 간의 유사성을 반영 X
 - ex) 원-핫 인코딩, TF-IDF
- **밀집 표현** (dense representation)
 - 단어를 고정된 크기의 실수 벡터로 표현
 - 단어 간의 거리를 효과적으로 계산
 - ex) Word2Vec
- **단어 임베딩 벡터**: 밀집 표현된 벡터
- Word2Vec은 밀집 표현을 위해 **CBow** 또는 **skip-gram** 방법을 사용한다.

[CBow]

- **CBoW** (Continuous Bag of Words): 주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법
 - **중심 단어**: 예측해야 할 단어
 - **주변 단어**: 예측에 사용되는 단어
 - **윈도(Window)**: 고려하는 주변 단어의 개수
 - 슬라이딩 윈도 사용 → 윈도를 이동해 가면서 학습
- 각 입력 단어의 원-핫 벡터를 **투사층**에 입력받고 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터를 예측함.
 1. 입력 단어는 원-핫 벡터로 표현돼 투사층에 입력
 2. 투사층은 **순람표 구조**로 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터 반환
→ $1 \times E$
 3. 입력된 단어들의 임베딩 벡터의 평균값 계산
 4. 평균 벡터를 가중치 행렬 W' ($E \times V$) 와 곱함 → $1 \times V$
 5. 벡터에 소프트맥스 함수를 이용해 중심 단어 예측
 - 투사층은 **룩업 테이블(LUT) 구조**로, 단어에 해당하는 임베딩 벡터를 찾아 반환한다.

[Skip-gram]

- **Skip-gram**: 중심 단어를 입력 받아 주변 단어를 예측하는 모델
 - 중심 단어를 기준으로 양쪽으로 윈도 크기만큼들의 단어들을 주변 단어로 삼아 훈련 데이터셋을 만든다.
 - Skip-gram vs CBoW:
 - CBoW는 하나의 윈도에서 하나의 학습 데이터가 만들어짐
 - Skip-gram은 중심 단어와 주변 단어를 하나의 쌍으로 해서 여러 학습 데이터가 만들어짐 → 더 많은 학습 데이터셋 추출
⇒ 더 뛰어난 성능을 보이며, 비교적 드물게 등장하는 단어를 더 잘 학습할 수 있게 됨
1. 입력 데이터의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터 가져옴
 2. 입력 단어의 임베딩과 W' ($E \times V$) 가중치를 곱해 V 크기의 벡터 얻음
 3. 벡터에 소프트맥스 연산을 취해 주변 단어를 예측

→ 소프트맥스 연산은 모든 단어를 대상으로 내적 연산을 수행하기 때문에 **학습 속도가 느려지는 단점**이 있다.

⇒ 계층적 소프트맥스와 네거티브 샘플링 기법 적용

[계층적 소프트맥스]

- **계층적 소프트맥스 (Hierarchical Softmax)**: 출력층을 **이진 트리 구조**로 표현
 - 자주 등장할수록 트리의 상위 노드에 위치
- 트리의 각 노드는 학습 가능한 벡터를 가짐 → 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 함
- 입력 벡터와 노드 벡터의 내적을 계산한 뒤 시그모이드 함수를 적용해 확률을 구함
- **잎 노드**: 트리의 가장 아래 노드, 각각 하나의 단어를 의미함
- 각 단어의 확률은 경로 노드의 확률을 곱해서 구함

[네거티브 샘플링]

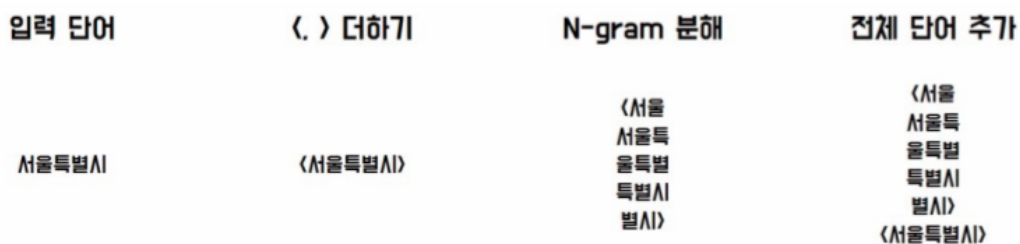
- **네거티브 샘플링 (Negative Sampling)**: Word2Vec 모델에서 사용되는 확률적인 샘플링 기법으로, 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용
- 학습 원도 내에 등장하지 않는 단어 n개(5~20) 추출 → 정답 단어와 함께 소프트맥스 연산 수행
- 각 단어가 네거티브 샘플로 추출될 확률:

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- $f(w)$: 각 단어의 출현 빈도수
- 0.75 제곱은 실험을 통해 얻어진 최적의 값
- **로지스틱 회귀 모델**을 사용해 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 네거티브 샘플링으로 생성된 단어 쌍인지 **이진 분류**
 1. 입력 단어의 임베딩과 레이블을 가져와 내적 연산 수행
 2. 시그모이드 함수를 통해 확률값으로 변환
 3. 레이블이 1인 경우 확률값이 높아지도록, 0인 경우 낮아지도록 모델의 가중치 최적화

2. fastText

- fastText: 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델로, Word2Vec와 유사하지만 N-gram을 사용해 단어의 하위 단어를 고려함
1. 단어의 벡터화를 위해 < >와 같은 특수 기호를 사용해 단어의 시작과 끝 나타냄
 2. N-gram을 사용해 하위 단어 집합(Subword set)으로 분해 (나뉘지지 않은 단어 자신도 포함)
 3. 각 하위 단어는 고유한 벡터값을 갖게 됨
 4. 각 하위 단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터 계산



- 같은 하위 단어를 공유하는 단어끼리는 정보를 공유해 학습할 수 있다
→ 비슷한 단어끼리는 비슷한 임베딩 벡터를 갖게 된다.
- OOV 단어도 하위 단어로 나눠 임베딩을 계산할 수 있다.

3. 순환 신경망

- 순환 신경망(Reccurent Neural Network, RNN): 순서가 있는 연속적인 데이터를 처리하는데, 이는 우리가 일상에서 하는 말들이 앞선 단어와 뒤따르는 단어 사이에 상관관계가 있다고 보기 때문임
1. 일대다 구조:
 - 하나의 입력 시퀀스, 여러 출력값
 - 문장에서 각 단어의 품사 예측이나 이미지 캡셔닝(Image Captioning) 모델에 활용

- 다만 출력 시퀀스 길이를 미리 알고 있어야 하며, 이를 위해 입력 시퀀스를 처리하면서 출력 시퀀스를 함께 예측하는 모델도 구현해야 한다.

2. 다대일 구조:

- 여러 개의 입력 시퀀스에 대해 하나의 출력값 생성.
- 문장의 긍정, 부정과 같은 감정 예측에 활용. 또는 입력 시퀀스의 범주를 구분하는 문장 분류, 문장 사이의 관계를 추론하는 자연어 추론 등에도 활용.

3. 다대다 구조:

- 여러 개의 입력 시퀀스를 여러 개의 출력 시퀀스로 뽑아내는 경우
- 대표적으로 번역기, 음성 인식 시스템과 같이 여러 입력 값을 받아 출력 값으로 뽑아내는 경우에 활용한다.
- 이러한 다대다 구조는 **시퀀스-시퀀스(Seq2Seq)**로, 두 RNN을 붙여 앞뒤로 각각 인코더(Encoder)와 디코더(Decoder)로 구성된다.

4. 양방향 순환 신경망:

- 기본적인 구조는 일반적인 RNN과 유사하지만, 이에 더해 시간 방향을 역방향으로도 처리할 수 있게 장치를 추가한 형태임
- 이를 통해 기존과 같이 $t-1, t, t+1$ 과 같이 시간 순서로 가던 신경망과 $t+1, t, t-1$ 과 같이 역방향으로 가는 신경망을 함께 고려한다.
- 이를 통해 "군대는 입대, (), 탈출로 이뤄져 있다."의 ()와 같이 앞뒤 문장을 모두 고려해야 하는 경우 양방향 순환 신경망을 통해 처리할 수 있다.

5. 다중 순환 신경망:

- RNN 층을 수직 방향으로 여러 층을 쌓은 형태
- 이를 통해 각 층에서 각각의 연산을 진행하면서 시간 방향으로 데이터를 넘기기도 하지만, 위에 있는 아예 다른 계층으로도 데이터를 넘길 수 있음
- 이와 같이 층을 깊게 하면 더 복잡한 패턴을 학습하는 것이 가능해짐
- 다만 이는 학습 시간의 증가와 기울기 소실 문제를 부를 수 있어 주의가 필요

4. 합성곱 신경망

- **합성곱 신경망 (CNN):** 컴퓨터 비전 분야의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류로, **합성곱 연산**을 사용함

- 특정 영역에서 출력 노드 생성 → 지역 특징 효과적 추출
- 이미지 데이터는 객체들의 위치와 형태가 자유분방하므로 여러 영역의 지역 특징을 조합 → 전역 특징 파악
- 순환 신경망은 병렬 처리가 어렵고 깊은 신경망을 구현하기 어려움 → 자연어 처리에도 합성곱 신경망이 사용되기 시작함

[합성곱 계층]

- 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층
- 고차원 데이터를 처리하는 데 주로 사용
- 입력 데이터의 모든 위치에서 동일한 필터 사용, 즉 모델 매개 변수 공유 → 과대 적합 방지

[활성화 맵]

- 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값
- 특징 맵에 **비선형성**을 추가하기 위해 활성화 함수를 적용
- 다음 계층의 입력값으로 사용

[풀링]

- 특징 맵의 크기를 줄이는 연산
- 연산량을 감소시키고 입력 데이터의 정보를 압축시킴
- 최댓값 풀링: 필터 내 원소값 중 가장 큰 값을 선택
- 평균값 풀링: 필터 내 원소값의 평균값 선택

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

[완전 연결 계층]

- 각 입력 노드가 모든 출력 노드와 연결된 상태
- 출력 노드의 수를 조절할 수 있으므로 모델의 복잡성과 용량을 조절하는 데 사용
- 일반적으로 특징 맵을 1차원 벡터로 변경하여 입력으로 받음
- 이전 계층에서 출력한 공간 정보는 무시되고 모든 입력을 독립적으로 처리

- 합성곱 신경망의 최종 출력을 결정